

Batch Normalization

1. 基本原理

Batch Normalization 是以进行学习时的 mini-batch 为单位，按 mini-batch 进行正规化的过程。具体而言，就是将每个特征在当前 mini-batch 上标准化，使其均值近似为 0、方差近似为 1。

2. 优点

使用 Batch Normalization 层有这几个优点：

- 可以使大部分学习快速进行（通常能够提高训练稳定性，并可能加快收敛速度，因此往往允许使用更大的学习率）。
- 不那么依赖初始值（降低模型对参数初始值的敏感程度）。
- 抑制过拟合。

3. 数学表达式

3.1 Mini-batch 的均值与方差

对于包含 m 个元素的 mini-batch，其均值为：

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

方差为：

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

对 mini-batch 中的元素进行正规化：

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

这里，我们对 mini-batch 中的 m 个元素进行正规化时需要在分母上加上微小量 ε ，防止分母为 0 的情况出现，同时也能在方差很小时提高数值的稳定性。

3.2 平移和缩放

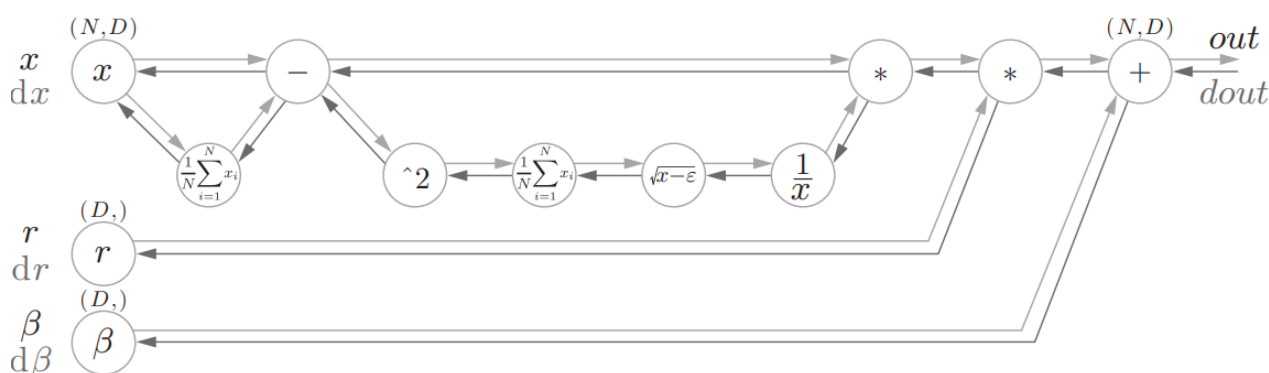
在对数据正规化后，我们对数据进行平移和缩放的变换，得到最后的输出结果，即：

$$y_i \leftarrow \gamma \hat{x}_i + \beta$$

其中， γ 和 β 是可调整参数，通常我们设置初始值为 $\gamma = 1$ 、 $\beta = 0$ ，后续通过学习调整至合适的值。

4. 计算图

Batch Normalization 层的计算图大致如下：



5. 反向传播

5.1 输入梯度公式

对于 Batch Normalization 层而言，反向传播的计算过程十分复杂，我们可以先只了解最后得到的计算式。针对于一个单个特征、含有 m 个样本的 mini-batch 而言，其反向传播的表达式为：

$$\frac{\partial L}{\partial x_i} = \frac{\gamma}{m\sqrt{\sigma_B^2 + \epsilon}} \left[mg_i - \sum_{j=1}^m g_j - \hat{x}_i \sum_{j=1}^m g_j \hat{x}_j \right]$$

同时， β 和 γ 的梯度分别为：

$$\frac{\partial L}{\partial \beta} = \sum_{j=1}^m g_j$$

$$\frac{\partial L}{\partial \gamma} = \sum_{j=1}^m g_j \hat{x}_j$$

参数说明

- m ：参与当前特征统计的元素数量，即 batch 的大小。
- γ ：可调整的缩放参数。
- g_j ：上游传回的梯度值。

5.2 完整的 Python 实现

若存在多个特征，即对每个特征参数同样处理即可，我们用 Python 代码完整地实现这个类：

```
class BatchNormalization:
    def __init__(self, gamma, beta, eps=1e-7):
```

```
self.gamma = gamma
self.beta = beta
self.eps = eps
self.cache = None
```

```
def forward(self, x):
```

```
# 每个特征顺着行的方向向下压，对列求均值，计算的是每个特征参数平均值
```

```
mu = np.mean(x, axis=0)
```

```
# 中心化，即计算每个值和均值的差，为后面算方差做准备
```

```
xc = x - mu
```

```
# 计算每个特征的方差，axis=0 和上面一样，表示顺着该矩阵的列方向计算
```

```
var = np.mean(xc**2, axis=0)
```

```
# 标准差的倒数
```

```
inv_std = 1.0 / np.sqrt(var + self.eps)
```

```
# 标准化
```

```
x_hat = xc * inv_std
```

```
# 缩放和平移
```

```
out = self.gamma * x_hat + self.beta
```

```
# 用元组存储反向传播需要使用的中间结果
```

```
self.cache = (xc, x_hat, inv_std)
```

```
return out
```

```
def backward(self, dout):
```

```
# 解包，获取到正向传播时已有的中间结果
```

```
xc, x_hat, inv_std = self.cache
```

```
m = dout.shape[0]
```

```
# 计算  $\beta(\text{beta})$  和  $\gamma(\text{gamma})$  的梯度
```

```
dbeta = np.sum(dout, axis=0)
```

```
dgamma = np.sum(dout * x_hat, axis=0)
```

```
#  $y = \gamma * x_{\text{hat}} + \beta$ 
```

```
dx_hat = dout * self.gamma
```

```
#  $x_{\text{hat}} = xc * inv\_std$ 
```

```
dx_c = dx_hat * inv_std
```

```
dinv_std = np.sum(dx_hat * xc, axis=0)
```

```
#  $inv\_std = (var + eps)^{-1/2}$ 
```

```
dvar = dinv_std * (-0.5) * inv_std**3
```

```
#  $var = \text{mean}(xc^2)$ 
```

```
dx_c += dvar * (2.0 / m) * xc
```

```

# xc = x - mu
dmu = -np.sum(dxc, axis=0)

# mu = mean(x)
dx = dxc + dmu / m

return dx, dgamma, dbeta

```

5.3 化简后的 Python 实现

当然，也可以用我们得到的简化计算式去简化反向传播的代码，提高计算效率。我们这里用 Python 简单实现一下这个功能，具体使用时需要略微改动一下，这里只做演示：

```

dx_hat = dout * gamma

dx = (
    inv_std
    / m
    * (
        m * dx_hat
        - np.sum(dx_hat, axis=0)
        - x_hat * np.sum(dx_hat * x_hat, axis=0)
    )
)

```